

6

DICTIONARIES



In this chapter you'll learn how to use Python's dictionaries, which allow you to connect pieces of related information. You'll learn how to access the information once it's in a dictionary and how to modify that information.

Because dictionaries can store an almost limitless amount of information, I'll show you how to loop through the data in a dictionary. Additionally, you'll learn to nest dictionaries inside lists, lists inside dictionaries, and even dictionaries inside other dictionaries.

Understanding dictionaries allows you to model a variety of real-world objects more accurately. You'll be able to create a dictionary representing a person and then store as much information as you want about that person. You can store their name, age, location, profession, and any other aspect of a person you can describe. You'll be able to store any two kinds of information that can be matched up, such as a list of words and their meanings, a list of people's names and their favorite numbers, a list of mountains and their elevations, and so forth.

A Simple Dictionary

Consider a game featuring aliens that can have different colors and point values. This simple dictionary stores information about a particular alien:

```
alien.py  alien_0 = {'color': 'green', 'points': 5}

          print(alien_0['color'])
          print(alien_0['points'])
```

The dictionary `alien_0` stores the alien's color and point value. The two `print` statements access and display that information, as shown here:

```
green
5
```

As with most new programming concepts, using dictionaries takes practice. Once you've worked with dictionaries for a bit you'll soon see how effectively they can model real-world situations.

Working with Dictionaries

A *dictionary* in Python is a collection of *key-value pairs*. Each *key* is connected to a value, and you can use a key to access the value associated with that key. A key's value can be a number, a string, a list, or even another dictionary. In fact, you can use any object that you can create in Python as a value in a dictionary.

In Python, a dictionary is wrapped in braces, `{}`, with a series of key-value pairs inside the braces, as shown in the earlier example:

```
alien_0 = {'color': 'green', 'points': 5}
```

A *key-value pair* is a set of values associated with each other. When you provide a key, Python returns the value associated with that key. Every key is connected to its value by a colon, and individual key-value pairs are separated by commas. You can store as many key-value pairs as you want in a dictionary.

The simplest dictionary has exactly one key-value pair, as shown in this modified version of the `alien_0` dictionary:

```
alien_0 = {'color': 'green'}
```

This dictionary stores one piece of information about `alien_0`, namely the alien's color. The string `'color'` is a key in this dictionary, and its associated value is `'green'`.

Accessing Values in a Dictionary

To get the value associated with a key, give the name of the dictionary and then place the key inside a set of square brackets, as shown here:

```
alien_0 = {'color': 'green'}  
print(alien_0['color'])
```

This returns the value associated with the key 'color' from the dictionary `alien_0`:

```
green
```

You can have an unlimited number of key-value pairs in a dictionary. For example, here's the original `alien_0` dictionary with two key-value pairs:

```
alien_0 = {'color': 'green', 'points': 5}
```

Now you can access either the color or the point value of `alien_0`. If a player shoots down this alien, you can look up how many points they should earn using code like this:

```
alien_0 = {'color': 'green', 'points': 5}
```

- ❶ `new_points = alien_0['points']`
 - ❷ `print("You just earned " + str(new_points) + " points!")`
-

Once the dictionary has been defined, the code at ❶ pulls the value associated with the key 'points' from the dictionary. This value is then stored in the variable `new_points`. The line at ❷ converts this integer value to a string and prints a statement about how many points the player just earned:

```
You just earned 5 points!
```

If you run this code every time an alien is shot down, the alien's point value will be retrieved.

Adding New Key-Value Pairs

Dictionaries are dynamic structures, and you can add new key-value pairs to a dictionary at any time. For example, to add a new key-value pair, you would give the name of the dictionary followed by the new key in square brackets along with the new value.

Let's add two new pieces of information to the `alien_0` dictionary: the alien's x- and y-coordinates, which will help us display the alien in a particular position on the screen. Let's place the alien on the left edge of the screen, 25 pixels down from the top. Because screen coordinates usually start at the upper-left corner of the screen, we'll place the alien on the left

edge of the screen by setting the x-coordinate to 0 and 25 pixels from the top by setting its y-coordinate to positive 25, as shown here:

```
alien_0 = {'color': 'green', 'points': 5}
print(alien_0)
```

- ❶ `alien_0['x_position'] = 0`
- ❷ `alien_0['y_position'] = 25`
`print(alien_0)`

We start by defining the same dictionary that we've been working with. We then print this dictionary, displaying a snapshot of its information. At ❶ we add a new key-value pair to the dictionary: key 'x_position' and value 0. We do the same for key 'y_position' at ❷. When we print the modified dictionary, we see the two additional key-value pairs:

```
{'color': 'green', 'points': 5}
{'color': 'green', 'points': 5, 'y_position': 25, 'x_position': 0}
```

The final version of the dictionary contains four key-value pairs. The original two specify color and point value, and two more specify the alien's position. Notice that the order of the key-value pairs does not match the order in which we added them. Python doesn't care about the order in which you store each key-value pair; it cares only about the connection between each key and its value.

Starting with an Empty Dictionary

It's sometimes convenient, or even necessary, to start with an empty dictionary and then add each new item to it. To start filling an empty dictionary, define a dictionary with an empty set of braces and then add each key-value pair on its own line. For example, here's how to build the `alien_0` dictionary using this approach:

```
alien_0 = {}

alien_0['color'] = 'green'
alien_0['points'] = 5

print(alien_0)
```

Here we define an empty `alien_0` dictionary, and then add color and point values to it. The result is the dictionary we've been using in previous examples:

```
{'color': 'green', 'points': 5}
```

Typically, you'll use empty dictionaries when storing user-supplied data in a dictionary or when you write code that generates a large number of key-value pairs automatically.

Modifying Values in a Dictionary

To modify a value in a dictionary, give the name of the dictionary with the key in square brackets and then the new value you want associated with that key. For example, consider an alien that changes from green to yellow as a game progresses:

```
alien_0 = {'color': 'green'}
print("The alien is " + alien_0['color'] + ".")

alien_0['color'] = 'yellow'
print("The alien is now " + alien_0['color'] + ".")
```

We first define a dictionary for `alien_0` that contains only the alien's color; then we change the value associated with the key 'color' to 'yellow'. The output shows that the alien has indeed changed from green to yellow:

```
The alien is green.
The alien is now yellow.
```

For a more interesting example, let's track the position of an alien that can move at different speeds. We'll store a value representing the alien's current speed and then use it to determine how far to the right the alien should move:

```
alien_0 = {'x_position': 0, 'y_position': 25, 'speed': 'medium'}
print("Original x-position: " + str(alien_0['x_position']))

# Move the alien to the right.
# Determine how far to move the alien based on its current speed.
❶ if alien_0['speed'] == 'slow':
    x_increment = 1
elif alien_0['speed'] == 'medium':
    x_increment = 2
else:
    # This must be a fast alien.
    x_increment = 3

# The new position is the old position plus the increment.
❷ alien_0['x_position'] = alien_0['x_position'] + x_increment

print("New x-position: " + str(alien_0['x_position']))
```

We start by defining an alien with an initial `x` position and `y` position, and a speed of 'medium'. We've omitted the color and point values for the sake of simplicity, but this example would work the same way if you included those key-value pairs as well. We also print the original value of `x_position` to see how far the alien moves to the right.

At ❶, an if-elif-else chain determines how far the alien should move to the right and stores this value in the variable `x_increment`. If the alien's speed is 'slow', it moves one unit to the right; if the speed is 'medium', it moves two

units to the right; and if it's 'fast', it moves three units to the right. Once the increment has been calculated, it's added to the value of `x_position` at ❷, and the result is stored in the dictionary's `x_position`.

Because this is a medium-speed alien, its position shifts two units to the right:

```
Original x-position: 0
New x-position: 2
```

This technique is pretty cool: by changing one value in the alien's dictionary, you can change the overall behavior of the alien. For example, to turn this medium-speed alien into a fast alien, you would add the line:

```
alien_0['speed'] = fast
```

The if-elif-else block would then assign a larger value to `x_increment` the next time the code runs.

Removing Key-Value Pairs

When you no longer need a piece of information that's stored in a dictionary, you can use the `del` statement to completely remove a key-value pair. All `del` needs is the name of the dictionary and the key that you want to remove.

For example, let's remove the key 'points' from the `alien_0` dictionary along with its value:

```
alien_0 = {'color': 'green', 'points': 5}
print(alien_0)
```

```
❶ del alien_0['points']
print(alien_0)
```

The line at ❶ tells Python to delete the key 'points' from the dictionary `alien_0` and to remove the value associated with that key as well. The output shows that the key 'points' and its value of 5 are deleted from the dictionary, but the rest of the dictionary is unaffected:

```
{'color': 'green', 'points': 5}
{'color': 'green'}
```

NOTE

Be aware that the deleted key-value pair is removed permanently.

A Dictionary of Similar Objects

The previous example involved storing different kinds of information about one object, an alien in a game. You can also use a dictionary to store one kind of information about many objects. For example, say you want to poll a

number of people and ask them what their favorite programming language is. A dictionary is useful for storing the results of a simple poll, like this:

```
favorite_languages = {
    'jen': 'python',
    'sarah': 'c',
    'edward': 'ruby',
    'phil': 'python',
}
```

As you can see, we've broken a larger dictionary into several lines. Each key is the name of a person who responded to the poll, and each value is their language choice. When you know you'll need more than one line to define a dictionary, press ENTER after the opening brace. Then indent the next line one level (four spaces), and write the first key-value pair, followed by a comma. From this point forward when you press ENTER, your text editor should automatically indent all subsequent key-value pairs to match the first key-value pair.

Once you've finished defining the dictionary, add a closing brace on a new line after the last key-value pair and indent it one level so it aligns with the keys in the dictionary. It's good practice to include a comma after the last key-value pair as well, so you're ready to add a new key-value pair on the next line.

NOTE

Most editors have some functionality that helps you format extended lists and dictionaries in a similar manner to this example. Other acceptable ways to format long dictionaries are available as well, so you may see slightly different formatting in your editor, or in other sources.

To use this dictionary, given the name of a person who took the poll, you can easily look up their favorite language:

favorite_
languages.py

```
favorite_languages = {
    'jen': 'python',
    'sarah': 'c',
    'edward': 'ruby',
    'phil': 'python',
}
```

```
❶ print("Sarah's favorite language is " +
❷     favorite_languages['sarah'].title() +
❸     ".")
```

To see which language Sarah chose, we ask for the value at:

```
favorite_languages['sarah']
```

This syntax is used in the print statement at ❷, and the output shows Sarah's favorite language:

```
Sarah's favorite language is C.
```

This example also shows how you can break up a long print statement over several lines. The word `print` is shorter than most dictionary names, so it makes sense to include the first part of what you want to print right after the opening parenthesis ❶. Choose an appropriate point at which to break what's being printed, and add a concatenation operator (+) at the end of the first line ❷. Press ENTER and then press TAB to align all subsequent lines at one indentation level under the print statement. When you've finished composing your output, you can place the closing parenthesis on the last line of the print block ❸.

TRY IT YOURSELF

6-1. Person: Use a dictionary to store information about a person you know. Store their first name, last name, age, and the city in which they live. You should have keys such as `first_name`, `last_name`, `age`, and `city`. Print each piece of information stored in your dictionary.

6-2. Favorite Numbers: Use a dictionary to store people's favorite numbers. Think of five names, and use them as keys in your dictionary. Think of a favorite number for each person, and store each as a value in your dictionary. Print each person's name and their favorite number. For even more fun, poll a few friends and get some actual data for your program.

6-3. Glossary: A Python dictionary can be used to model an actual dictionary. However, to avoid confusion, let's call it a glossary.

- Think of five programming words you've learned about in the previous chapters. Use these words as the keys in your glossary, and store their meanings as values.
- Print each word and its meaning as neatly formatted output. You might print the word followed by a colon and then its meaning, or print the word on one line and then print its meaning indented on a second line. Use the newline character (`\n`) to insert a blank line between each word-meaning pair in your output.

Looping Through a Dictionary

A single Python dictionary can contain just a few key-value pairs or millions of pairs. Because a dictionary can contain large amounts of data, Python lets you loop through a dictionary. Dictionaries can be used to store information in a variety of ways; therefore, several different ways exist to loop through them. You can loop through all of a dictionary's key-value pairs, through its keys, or through its values.

Looping Through All Key-Value Pairs

Before we explore the different approaches to looping, let's consider a new dictionary designed to store information about a user on a website. The following dictionary would store one person's username, first name, and last name:

```
user_0 = {  
    'username': 'efermi',  
    'first': 'enrico',  
    'last': 'fermi',  
}
```

You can access any single piece of information about `user_0` based on what you've already learned in this chapter. But what if you wanted to see everything stored in this user's dictionary? To do so, you could loop through the dictionary using a for loop:

```
user.py  user_0 = {  
          'username': 'efermi',  
          'first': 'enrico',  
          'last': 'fermi',  
        }
```

- ❶ for key, value in user_0.items():
 - ❷ print("\nKey: " + key)
 - ❸ print("Value: " + value)
-

As shown at ❶, to write a for loop for a dictionary, you create names for the two variables that will hold the key and value in each key-value pair. You can choose any names you want for these two variables. This code would work just as well if you had used abbreviations for the variable names, like this:

```
for k, v in user_0.items()
```

The second half of the for statement at ❶ includes the name of the dictionary followed by the method `items()`, which returns a list of key-value pairs. The for loop then stores each of these pairs in the two variables provided. In the preceding example, we use the variables to print each key ❷, followed by the associated value ❸. The `"\n"` in the first print statement ensures that a blank line is inserted before each key-value pair in the output:

```
Key: last  
Value: fermi
```

```
Key: first  
Value: enrico
```

```
Key: username  
Value: efermi
```

Notice again that the key-value pairs are not returned in the order in which they were stored, even when looping through a dictionary. Python doesn't care about the order in which key-value pairs are stored; it tracks only the connections between individual keys and their values.

Looping through all key-value pairs works particularly well for dictionaries like the *favorite_languages.py* example on page 101, which stores the same kind of information for many different keys. If you loop through the *favorite_languages* dictionary, you get the name of each person in the dictionary and their favorite programming language. Because the keys always refer to a person's name and the value is always a language, we'll use the variables *name* and *language* in the loop instead of *key* and *value*. This will make it easier to follow what's happening inside the loop:

favorite_
languages.py

```
favorite_languages = {  
    'jen': 'python',  
    'sarah': 'c',  
    'edward': 'ruby',  
    'phil': 'python',  
}
```

- ```
❶ for name, language in favorite_languages.items():
❷ print(name.title() + "'s favorite language is " +
 language.title() + ".")
```
- 

The code at ❶ tells Python to loop through each key-value pair in the dictionary. As it works through each pair the key is stored in the variable *name*, and the value is stored in the variable *language*. These descriptive names make it much easier to see what the print statement at ❷ is doing.

Now, in just a few lines of code, we can display all of the information from the poll:

---

```
Jen's favorite language is Python.
Sarah's favorite language is C.
Phil's favorite language is Python.
Edward's favorite language is Ruby.
```

---

This type of looping would work just as well if our dictionary stored the results from polling a thousand or even a million people.

## ***Looping Through All the Keys in a Dictionary***

The *keys()* method is useful when you don't need to work with all of the values in a dictionary. Let's loop through the *favorite\_languages* dictionary and print the names of everyone who took the poll:

---

```
favorite_languages = {
 'jen': 'python',
 'sarah': 'c',
 'edward': 'ruby',
 'phil': 'python',
}
```

```
❶ for name in favorite_languages.keys():
 print(name.title())
```

---

The line at ❶ tells Python to pull all the keys from the dictionary `favorite_languages` and store them one at a time in the variable `name`. The output shows the names of everyone who took the poll:

---

```
Jen
Sarah
Phil
Edward
```

---

Looping through the keys is actually the default behavior when looping through a dictionary, so this code would have exactly the same output if you wrote ...

---

```
for name in favorite_languages:
```

---

rather than ...

---

```
for name in favorite_languages.keys():
```

---

You can choose to use the `keys()` method explicitly if it makes your code easier to read, or you can omit it if you wish.

You can access the value associated with any key you care about inside the loop by using the current key. Let's print a message to a couple of friends about the languages they chose. We'll loop through the names in the dictionary as we did previously, but when the name matches one of our friends, we'll display a message about their favorite language:

---

```
favorite_languages = {
 'jen': 'python',
 'sarah': 'c',
 'edward': 'ruby',
 'phil': 'python',
}
```

```
❶ friends = ['phil', 'sarah']
for name in favorite_languages.keys():
 print(name.title())

❷ if name in friends:
 print(" Hi " + name.title() +
 ", I see your favorite language is " +
❸ favorite_languages[name].title() + "!")
```

---

At ❶ we make a list of friends that we want to print a message to. Inside the loop, we print each person's name. Then at ❷ we check to see whether the name we are working with is in the list `friends`. If it is, we print a special greeting, including a reference to their language choice. To access

the favorite language at ❸, we use the name of the dictionary and the current value of `name` as the key. Everyone's name is printed, but our friends receive a special message:

---

```
Edward
Phil
 Hi Phil, I see your favorite language is Python!
Sarah
 Hi Sarah, I see your favorite language is C!
Jen
```

---

You can also use the `keys()` method to find out if a particular person was polled. This time, let's find out if Erin took the poll:

---

```
favorite_languages = {
 'jen': 'python',
 'sarah': 'c',
 'edward': 'ruby',
 'phil': 'python',
}
```

```
❶ if 'erin' not in favorite_languages.keys():
 print("Erin, please take our poll!")
```

---

The `keys()` method isn't just for looping: It actually returns a list of all the keys, and the line at ❶ simply checks if 'erin' is in this list. Because she's not, a message is printed inviting her to take the poll:

---

```
Erin, please take our poll!
```

---

## ***Looping Through a Dictionary's Keys in Order***

A dictionary always maintains a clear connection between each key and its associated value, but you never get the items from a dictionary in any predictable order. That's not a problem, because you'll usually just want to obtain the correct value associated with each key.

One way to return items in a certain order is to sort the keys as they're returned in the for loop. You can use the `sorted()` function to get a copy of the keys in order:

---

```
favorite_languages = {
 'jen': 'python',
 'sarah': 'c',
 'edward': 'ruby',
 'phil': 'python',
}
```

```
for name in sorted(favorite_languages.keys()):
 print(name.title() + ", thank you for taking the poll.")
```

---

This for statement is like other for statements except that we've wrapped the `sorted()` function around the `dictionary.keys()` method. This tells Python to list all keys in the dictionary and sort that list before looping through it. The output shows everyone who took the poll with the names displayed in order:

---

```
Edward, thank you for taking the poll.
Jen, thank you for taking the poll.
Phil, thank you for taking the poll.
Sarah, thank you for taking the poll.
```

---

## ***Looping Through All Values in a Dictionary***

If you are primarily interested in the values that a dictionary contains, you can use the `values()` method to return a list of values without any keys. For example, say we simply want a list of all languages chosen in our programming language poll without the name of the person who chose each language:

---

```
favorite_languages = {
 'jen': 'python',
 'sarah': 'c',
 'edward': 'ruby',
 'phil': 'python',
}

print("The following languages have been mentioned:")
for language in favorite_languages.values():
 print(language.title())
```

---

The for statement here pulls each value from the dictionary and stores it in the variable `language`. When these values are printed, we get a list of all chosen languages:

---

```
The following languages have been mentioned:
Python
C
Python
Ruby
```

---

This approach pulls all the values from the dictionary without checking for repeats. That might work fine with a small number of values, but in a poll with a large number of respondents, this would result in a very repetitive list. To see each language chosen without repetition, we can use a *set*. A set is similar to a list except that each item in the set must be unique:

---

```
favorite_languages = {
 'jen': 'python',
 'sarah': 'c',
 'edward': 'ruby',
```

```

 'phil': 'python',
 }

 print("The following languages have been mentioned:")
 ❶ for language in set(favorite_languages.values()):
 print(language.title())

```

When you wrap `set()` around a list that contains duplicate items, Python identifies the unique items in the list and builds a set from those items. At ❶ we use `set()` to pull out the unique languages in `favorite_languages.values()`.

The result is a nonrepetitive list of languages that have been mentioned by people taking the poll:

---

```

The following languages have been mentioned:
Python
C
Ruby

```

---

As you continue learning about Python, you'll often find a built-in feature of the language that helps you do exactly what you want with your data.

### TRY IT YOURSELF

**6-4. Glossary 2:** Now that you know how to loop through a dictionary, clean up the code from Exercise 6-3 (page 102) by replacing your series of `print` statements with a loop that runs through the dictionary's keys and values. When you're sure that your loop works, add five more Python terms to your glossary. When you run your program again, these new words and meanings should automatically be included in the output.

**6-5. Rivers:** Make a dictionary containing three major rivers and the country each river runs through. One key-value pair might be `'nile': 'egypt'`.

- Use a loop to print a sentence about each river, such as *The Nile runs through Egypt*.
- Use a loop to print the name of each river included in the dictionary.
- Use a loop to print the name of each country included in the dictionary.

**6-6. Polling:** Use the code in `favorite_languages.py` (page 104).

- Make a list of people who should take the favorite languages poll. Include some names that are already in the dictionary and some that are not.
- Loop through the list of people who should take the poll. If they have already taken the poll, print a message thanking them for responding. If they have not yet taken the poll, print a message inviting them to take the poll.

## Nesting

Sometimes you'll want to store a set of dictionaries in a list or a list of items as a value in a dictionary. This is called *nesting*. You can nest a set of dictionaries inside a list, a list of items inside a dictionary, or even a dictionary inside another dictionary. Nesting is a powerful feature, as the following examples will demonstrate.

### A List of Dictionaries

The `alien_0` dictionary contains a variety of information about one alien, but it has no room to store information about a second alien, much less a screen full of aliens. How can you manage a fleet of aliens? One way is to make a list of aliens in which each alien is a dictionary of information about that alien. For example, the following code builds a list of three aliens:

```
aliens.py alien_0 = {'color': 'green', 'points': 5}
alien_1 = {'color': 'yellow', 'points': 10}
alien_2 = {'color': 'red', 'points': 15}

❶ aliens = [alien_0, alien_1, alien_2]

for alien in aliens:
 print(alien)
```

We first create three dictionaries, each representing a different alien. At ❶ we pack each of these dictionaries into a list called `aliens`. Finally, we loop through the list and print out each alien:

```
{'color': 'green', 'points': 5}
{'color': 'yellow', 'points': 10}
{'color': 'red', 'points': 15}
```

A more realistic example would involve more than three aliens with code that automatically generates each alien. In the following example we use `range()` to create a fleet of 30 aliens:

```
Make an empty list for storing aliens.
aliens = []

Make 30 green aliens.
❶ for alien_number in range(30):
❷ new_alien = {'color': 'green', 'points': 5, 'speed': 'slow'}
❸ aliens.append(new_alien)

Show the first 5 aliens:
❹ for alien in aliens[:5]:
 print(alien)
 print("...")

Show how many aliens have been created.
❺ print("Total number of aliens: " + str(len(aliens)))
```

This example begins with an empty list to hold all of the aliens that will be created. At ❶ `range()` returns a set of numbers, which just tells Python how many times we want the loop to repeat. Each time the loop runs we create a new alien ❷ and then append each new alien to the list aliens ❸. At ❹ we use a slice to print the first five aliens, and then at ❺ we print the length of the list to prove we’ve actually generated the full fleet of 30 aliens:

---

```
{'speed': 'slow', 'color': 'green', 'points': 5}
{'speed': 'slow', 'color': 'green', 'points': 5}
{'speed': 'slow', 'color': 'green', 'points': 5}
{'speed': 'slow', 'color': 'green', 'points': 5}
{'speed': 'slow', 'color': 'green', 'points': 5}
...
```

```
Total number of aliens: 30
```

---

These aliens all have the same characteristics, but Python considers each one a separate object, which allows us to modify each alien individually.

How might you work with a set of aliens like this? Imagine that one aspect of a game has some aliens changing color and moving faster as the game progresses. When it’s time to change colors, we can use a `for` loop and an `if` statement to change the color of aliens. For example, to change the first three aliens to yellow, medium-speed aliens worth 10 points each, we could do this:

---

```
Make an empty list for storing aliens.
aliens = []

Make 30 green aliens.
for alien_number in range(0,30):
 new_alien = {'color': 'green', 'points': 5, 'speed': 'slow'}
 aliens.append(new_alien)

for alien in aliens[0:3]:
 if alien['color'] == 'green':
 alien['color'] = 'yellow'
 alien['speed'] = 'medium'
 alien['points'] = 10

Show the first 5 aliens:
for alien in aliens[0:5]:
 print(alien)
print("...")
```

---

Because we want to modify the first three aliens, we loop through a slice that includes only the first three aliens. All of the aliens are green now but that won’t always be the case, so we write an `if` statement to make sure



we're only modifying green aliens. If the alien is green, we change the color to 'yellow', the speed to 'medium', and the point value to 10, as shown in the following output:

---

```
{'speed': 'medium', 'color': 'yellow', 'points': 10}
{'speed': 'medium', 'color': 'yellow', 'points': 10}
{'speed': 'medium', 'color': 'yellow', 'points': 10}
{'speed': 'slow', 'color': 'green', 'points': 5}
{'speed': 'slow', 'color': 'green', 'points': 5}
...
```

---

You could expand this loop by adding an `elif` block that turns yellow aliens into red, fast-moving ones worth 15 points each. Without showing the entire program again, that loop would look like this:

---

```
for alien in aliens[0:3]:
 if alien['color'] == 'green':
 alien['color'] = 'yellow'
 alien['speed'] = 'medium'
 alien['points'] = 10
 elif alien['color'] == 'yellow':
 alien['color'] = 'red'
 alien['speed'] = 'fast'
 alien['points'] = 15
```

---

It's common to store a number of dictionaries in a list when each dictionary contains many kinds of information about one object. For example, you might create a dictionary for each user on a website, as we did in *user.py* on page 103, and store the individual dictionaries in a list called `users`. All of the dictionaries in the list should have an identical structure so you can loop through the list and work with each dictionary object in the same way.

## A List in a Dictionary

Rather than putting a dictionary inside a list, it's sometimes useful to put a list inside a dictionary. For example, consider how you might describe a pizza that someone is ordering. If you were to use only a list, all you could really store is a list of the pizza's toppings. With a dictionary, a list of toppings can be just one aspect of the pizza you're describing.

In the following example, two kinds of information are stored for each pizza: a type of crust and a list of toppings. The list of toppings is a value associated with the key `'toppings'`. To use the items in the list, we give the name of the dictionary and the key `'toppings'`, as we would any value in the dictionary. Instead of returning a single value, we get a list of toppings:

---

```
pizza.py # Store information about a pizza being ordered.
❶ pizza = {
 'crust': 'thick',
 'toppings': ['mushrooms', 'extra cheese'],
}
```

```

Summarize the order.
❷ print("You ordered a " + pizza['crust'] + "-crust pizza " +
 "with the following toppings:")

❸ for topping in pizza['toppings']:
 print("\t" + topping)

```

---

We begin at ❶ with a dictionary that holds information about a pizza that has been ordered. One key in the dictionary is 'crust', and the associated value is the string 'thick'. The next key, 'toppings', has a list as its value that stores all requested toppings. At ❷ we summarize the order before building the pizza. To print the toppings, we write a for loop ❸. To access the list of toppings, we use the key 'toppings', and Python grabs the list of toppings from the dictionary.

The following output summarizes the pizza that we plan to build:

---

```

You ordered a thick-crust pizza with the following toppings:
mushrooms
extra cheese

```

---

You can nest a list inside a dictionary any time you want more than one value to be associated with a single key in a dictionary. In the earlier example of favorite programming languages, if we were to store each person's responses in a list, people could choose more than one favorite language. When we loop through the dictionary, the value associated with each person would be a list of languages rather than a single language. Inside the dictionary's for loop, we use another for loop to run through the list of languages associated with each person:

```

favorite_ ❶ favorite_languages = {
languages.py 'jen': ['python', 'ruby'],
 'sarah': ['c'],
 'edward': ['ruby', 'go'],
 'phil': ['python', 'haskell'],
 }

❷ for name, languages in favorite_languages.items():
 print("\n" + name.title() + "'s favorite languages are:")
❸ for language in languages:
 print("\t" + language.title())

```

---

As you can see at ❶ the value associated with each name is now a list. Notice that some people have one favorite language and others have multiple favorites. When we loop through the dictionary at ❷, we use the variable name `languages` to hold each value from the dictionary, because we know that each value will be a list. Inside the main dictionary loop, we use

another for loop ❸ to run through each person's list of favorite languages. Now each person can list as many favorite languages as they like:

---

Jen's favorite languages are:

Python  
Ruby

Sarah's favorite languages are:

C

Phil's favorite languages are:

Python  
Haskell

Edward's favorite languages are:

Ruby  
Go

---

To refine this program even further, you could include an if statement at the beginning of the dictionary's for loop to see whether each person has more than one favorite language by examining the value of `len(languages)`. If a person has more than one favorite, the output would stay the same. If the person has only one favorite language, you could change the wording to reflect that. For example, you could say Sarah's favorite language is C.

**NOTE**

*You should not nest lists and dictionaries too deeply. If you're nesting items much deeper than what you see in the preceding examples or you're working with someone else's code with significant levels of nesting, most likely a simpler way to solve the problem exists.*

## ***A Dictionary in a Dictionary***

You can nest a dictionary inside another dictionary, but your code can get complicated quickly when you do. For example, if you have several users for a website, each with a unique username, you can use the usernames as the keys in a dictionary. You can then store information about each user by using a dictionary as the value associated with their username. In the following listing, we store three pieces of information about each user: their first name, last name, and location. We'll access this information by looping through the usernames and the dictionary of information associated with each username:

---

*many\_users.py*

```
users = {
 'aeinstein': {
 'first': 'albert',
 'last': 'einstein',
 'location': 'princeton',
 },
}
```

```
'mcurie': {
 'first': 'marie',
 'last': 'curie',
 'location': 'paris',
},

}
```

```
❶ for username, user_info in users.items():
❷ print("\nUsername: " + username)
❸ full_name = user_info['first'] + " " + user_info['last']
 location = user_info['location']

❹ print("\tFull name: " + full_name.title())
 print("\tLocation: " + location.title())
```

---

We first define a dictionary called `users` with two keys: one each for the usernames `'aeinstein'` and `'mcurie'`. The value associated with each key is a dictionary that includes each user's first name, last name, and location. At ❶ we loop through the `users` dictionary. Python stores each key in the variable `username`, and the dictionary associated with each username goes into the variable `user_info`. Once inside the main dictionary loop, we print the username at ❷.

At ❸ we start accessing the inner dictionary. The variable `user_info`, which contains the dictionary of user information, has three keys: `'first'`, `'last'`, and `'location'`. We use each key to generate a neatly formatted full name and location for each person, and then print a summary of what we know about each user ❹:

---

```
Username: aeinstein
 Full name: Albert Einstein
 Location: Princeton

Username: mcurie
 Full name: Marie Curie
 Location: Paris
```

---

Notice that the structure of each user's dictionary is identical. Although not required by Python, this structure makes nested dictionaries easier to work with. If each user's dictionary had different keys, the code inside the `for` loop would be more complicated.

### TRY IT YOURSELF

**6-7. People:** Start with the program you wrote for Exercise 6-1 (page 102). Make two new dictionaries representing different people, and store all three dictionaries in a list called `people`. Loop through your list of `people`. As you loop through the list, print everything you know about each person.

**6-8. Pets:** Make several dictionaries, where the name of each dictionary is the name of a pet. In each dictionary, include the kind of animal and the owner's name. Store these dictionaries in a list called `pets`. Next, loop through your list and as you do print everything you know about each pet.

**6-9. Favorite Places:** Make a dictionary called `favorite_places`. Think of three names to use as keys in the dictionary, and store one to three favorite places for each person. To make this exercise a bit more interesting, ask some friends to name a few of their favorite places. Loop through the dictionary, and print each person's name and their favorite places.

**6-10. Favorite Numbers:** Modify your program from Exercise 6-2 (page 102) so each person can have more than one favorite number. Then print each person's name along with their favorite numbers.

**6-11. Cities:** Make a dictionary called `cities`. Use the names of three cities as keys in your dictionary. Create a dictionary of information about each city and include the country that the city is in, its approximate population, and one fact about that city. The keys for each city's dictionary should be something like `country`, `population`, and `fact`. Print the name of each city and all of the information you have stored about it.

**6-12. Extensions:** We're now working with examples that are complex enough that they can be extended in any number of ways. Use one of the example programs from this chapter, and extend it by adding new keys and values, changing the context of the program or improving the formatting of the output.

## Summary

In this chapter you learned how to define a dictionary and how to work with the information stored in a dictionary. You learned how to access and modify individual elements in a dictionary, and how to loop through all of the information in a dictionary. You learned to loop through a dictionary's key-value pairs, its keys, and its values. You also learned how to nest multiple dictionaries in a list, nest lists in a dictionary, and nest a dictionary inside a dictionary.

In the next chapter you'll learn about `while` loops and how to accept input from people who are using your programs. This will be an exciting chapter, because you'll learn to make all of your programs interactive: they'll be able to respond to user input.